



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

You Mean There's More?

Congratulations! You've reached the end of your journey through the landscape of game engine architecture in one piece (and hopefully none the worse for wear). With any luck, you've learned a great deal about the major components that comprise a typical game engine. But of course, every journey's end is another's beginning. There's a great deal more to be learned about each and every topic covered within these pages. As technology and computing hardware continue to improve, more things will become possible in games—and more engine systems will be invented to support them. What's more, this book's focus was on the game engine itself. We haven't even begun to discuss the rich world of gameplay programming, a topic that could fill many more volumes.

In the following brief sections, I'll identify a few of the engine and gameplay systems we didn't have room to cover in any depth in this book, and I'll suggest some resources for those who wish to learn more about them.

17.1 Some Engine Systems We Didn't Cover

17.1.1 Movie Player

Most games include a movie player for displaying prerendered movies, also known as full-motion video (FMV). The basic components of the movie player

are an interface to the streaming file I/O system (see Section 7.1.3), a codec to decode the compressed video stream, and some form of synchronization with the audio playback system for the sound track.

A number of different video encoding standards and corresponding codecs are available, each one suited to a particular type of application. For example, video CDs (VCD) and DVDs use MPEG-1 and MPEG-2 (H.262) codecs, respectively. The H.261 and H.263 standards are designed primarily for online video conferencing applications. Games often use standards like MPEG-4 part 2 (e.g., DivX), MPEG-4 Part 10 / H.264, Windows Media Video (WMV) or Bink Video (a standard designed specifically for games by Rad Game Tools, Inc.). See http://en.wikipedia.org/wiki/Video_codec and <http://www.radgame.com/bnkmain.htm> for more information on video codecs.

17.1.2 Multiplayer Networking

Although the concepts of concurrent programming covered in Chapter 4 are relevant to multiplayer game architecture and distributed network programming, this book doesn't address either topic directly. For an in-depth treatment of multiplayer networking, see [4].

17.2 Gameplay Systems

A game is of course much more than just its engine. On top of the gameplay foundation layer (discussed in Chapter 16), you'll find a rich assortment of genre- and game-specific gameplay systems. These systems tie the myriad game engine technologies described in this book together into a cohesive whole, breathing life into the game.

17.2.1 Player Mechanics

Player mechanics are of course the most important gameplay system. Each genre is defined by a general style of player mechanics and gameplay, and of course every game within a genre has its own specific designs. As such, player mechanics is a huge topic. It involves the integration of human interface device systems, motion simulation, collision detection, animation and audio, not to mention integration with other gameplay systems like the game camera, weapons, cover, specialized traversal mechanics (ladders, swinging ropes, etc.), vehicle systems, puzzle mechanics and so on.

Clearly, player mechanics are as varied as the games themselves, so there's no one place you can go to learn all about them. It's best to tackle this topic by studying a single genre at a time. Play games and try to reverse-engineer

their player mechanics. Then try to implement them yourself! And as a very modest start on reading, you can check out [9, Section 4.11] for a discussion of Mario-style platformer player mechanics.

17.2.2 Cameras

A game's camera system is almost as important as the player mechanics. In fact, the camera can make or break the gameplay experience. Each genre tends to have its own camera control style, although of course every game within a particular genre does it a little bit differently (and some *very* differently). See [8, Section 4.3] for some basic game camera control techniques. In the following paragraphs, I'll briefly outline some of the most prevalent kinds of cameras in 3D games, but please note that this is far from a complete list.

- *Look-at cameras.* This type of camera rotates about a target point and can be moved in and out relative to this point.
- *Follow cameras.* This type of camera is prevalent in platformer, third-person shooter and vehicle-based games. It acts much like a look-at camera focused on the player character/avatar/vehicle, but its motion typically lags the player. A follow camera also includes advanced collision detection and avoidance logic and provides the human player with some degree of control over the camera's orientation relative to the player avatar.
- *First-person cameras.* As the player character moves about in the game world, a first-person camera remains affixed to the character's virtual eyes. The player typically has full control over the direction in which the camera should be pointed, either via mouse or joypad control. The look direction of the camera also translates directly into the aim direction of the player's weapon, which is typically indicated by a set of disembodied arms and a gun attached to the bottom of the screen, and a reticle at the center of the screen.
- *RTS cameras.* Real-time strategy and god games tend to employ a camera that floats above the terrain, looking down at an angle. The camera can be panned about over the terrain, but the pitch and yaw of the camera are usually not under direct player control.
- *Cinematic cameras.* Most three-dimensional games have at least some cinematic moments in which the camera flies about within the scene in a more filmic manner rather than being tethered to an object in the game. These camera movements are typically controlled by the animators.

17.2.3 Artificial Intelligence

Another major component of most character-based games is *artificial intelligence* (AI). At its lowest level, an AI system is usually founded in technologies like basic path finding (which commonly makes use of the well-known A* algorithm), perception systems (line of sight, vision cones, knowledge of the environment, etc.) and some form of memory or knowledge.

On top of these foundations, character control logic is implemented. A character control system determines how to make the character perform specific actions like locomoting, navigating unusual terrain features, using weapons, driving vehicles, taking cover and so on. It typically involves complex interfaces to the collision, physics and animation systems within the engine. Character control is discussed in detail in Section 12.10.

Above the character control layer, an AI system typically has goal setting and decision making logic, and possibly also emotional state modeling, group behaviors (coordination, flanking, crowd and flocking behaviors, etc.), and perhaps some advanced features like an ability to learn from past mistakes or adapt to a changing environment.

Of course, the term “artificial intelligence” is one of the biggest misnomers around in the game industry. Game AI is always more of a smoke and mirrors job than an attempt at truly mimicking human intelligence. Your AI characters might have all sorts of complex internal emotional states and finely tuned perception of the game world. But if the player cannot *perceive* the characters’ motivations, it’s all for naught.

AI programming is a rich topic, and we certainly have not done it justice in this book. For more information, see [18], [8, Section 3], [9, Section 3] and [47, Section 3]. Another good starting point is the GDC 2002 talk entitled, “The Illusion of Intelligence: The Integration of AI and Level Design in Halo,” by Chris Butcher and Jaime Griesemer of Bungie (<http://bit.ly/1g7FbhD>). And while you’re online, search for “game AI programming” too. You’ll find all sorts of links to talks, papers and books on game AI. The websites <http://aigamedev.com> and <http://www.gameai.com> are great resources as well.

17.2.4 Other Gameplay Systems

Clearly there’s a lot more to a game than just player mechanics, cameras and AI. Some games have drivable vehicles, implement specialized types of weaponry, allow the player to destroy the environment with the help of a dynamic physics simulation, let the player create his or her own characters, build custom levels, require the player to solve puzzles, Of course, the list of genre- and game-specific features, and all of the specialized software systems

that implement them, could go on forever. Gameplay systems are as rich and varied as games are. Perhaps this is where your next journey as a game programmer will begin!



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Bibliography

- [1] Michael Abrash. *Michael Abrash's Graphics Programming Black Book (Special Edition)*. Scottsdale, AZ: Coriolis Group Books, 1997. (Available online at <http://www.jagregory.com/abrash-black-book>.)
- [2] Tomas Akenine-Moller, Eric Haines and Naty Hoffman. *Real-Time Rendering*, Third Edition. Wellesley, MA: A K Peters, 2008.
- [3] Andrei Alexandrescu. *Modern C++ Design: Generic Programming and Design Patterns Applied*. Reading, MA: Addison-Wesley, 2001.
- [4] Grenville Armitage, Mark Claypool and Philip Branch. *Networking and Online Games: Understanding and Engineering Multiplayer Internet Games*. New York, NY: John Wiley and Sons, 2006.
- [5] James Arvo (editor). *Graphics Gems II*. San Diego, CA: Academic Press, 1991.
- [6] David A. Bies and Colin H. Hansen. *Engineering Noise Control*, Fourth Edition. New York, NY: CRC Press, 2014.
- [7] Grady Booch, Robert A. Maksimchuk, Michael W. Engel, Bobbi J. Young, Jim Conallen and Kelli A. Houston. *Object-Oriented Analysis and Design with Applications*, Third Edition. Reading, MA: Addison-Wesley, 2007.
- [8] Mark DeLoura (editor). *Game Programming Gems*. Hingham, MA: Charles River Media, 2000.
- [9] Mark DeLoura (editor). *Game Programming Gems 2*. Hingham, MA: Charles River Media, 2001.

- [10] Philip Dutré, Kavita Bala and Philippe Bekaert. *Advanced Global Illumination*, Second Edition. Wellesley, MA: A K Peters, 2006.
- [11] David H. Eberly. *3D Game Engine Design: A Practical Approach to Real-Time Computer Graphics*. San Francisco, CA: Morgan Kaufmann, 2001.
- [12] David H. Eberly. *3D Game Engine Architecture: Engineering Real-Time Applications with Wild Magic*. San Francisco, CA: Morgan Kaufmann, 2005.
- [13] David H. Eberly. *Game Physics*. San Francisco, CA: Morgan Kaufmann, 2003.
- [14] Christer Ericson. *Real-Time Collision Detection*. San Francisco, CA: Morgan Kaufmann, 2005.
- [15] Randima Fernando (editor). *GPU Gems: Programming Techniques, Tips and Tricks for Real-Time Graphics*. Reading, MA: Addison-Wesley, 2004.
- [16] James D. Foley, Andries van Dam, Steven K. Feiner and John F. Hughes. *Computer Graphics: Principles and Practice in C*, Second Edition. Reading, MA: Addison-Wesley, 1995.
- [17] Grant R. Fowles and George L. Cassiday. *Analytical Mechanics*, Seventh Edition. Pacific Grove, CA: Brooks Cole, 2005.
- [18] John David Funge. *AI for Games and Animation: A Cognitive Modeling Approach*. Wellesley, MA: A K Peters, 1999.
- [19] Erich Gamma, Richard Helm, Ralph Johnson and John M. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.
- [20] Andrew S. Glassner (editor). *Graphics Gems I*. San Francisco, CA: Morgan Kaufmann, 1990.
- [21] Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar. *Introduction to Parallel Computing*, Second Edition. Reading, MA: Addison Wesley, 2003. (Available online at http://srmcse.weebly.com/uploads/8/9/0/9/8909020/introduction_to_parallel_computing_second_edition-ananth_grama..pdf [sic].)
- [22] Paul S. Heckbert (editor). *Graphics Gems IV*. San Diego, CA: Academic Press, 1994.
- [23] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. San Francisco, CA: Morgan Kaufmann, 2011.
- [24] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. San Francisco, CA: Morgan Kaufmann, 2008.
- [25] Roberto Ierusalimschy, Luiz Henrique de Figueiredo and Waldemar Celes. *Lua 5.1 Reference Manual*. Lua.org, 2006.
- [26] Roberto Ierusalimschy. *Programming in Lua*, Second Edition. Lua.org, 2006.
- [27] Isaac Victor Kerlow. *The Art of 3-D Computer Animation and Imaging (Second Edition)*. New York, NY: John Wiley and Sons, 2000.
- [28] David Kirk (editor). *Graphics Gems III*. San Francisco, CA: Morgan Kaufmann, 1994.

- [29] Danny Kodicek. *Mathematics and Physics for Game Programmers*. Hingham, MA: Charles River Media, 2005.
- [30] Raph Koster. *A Theory of Fun for Game Design*. Phoenix, AZ: Paraglyph, 2004.
- [31] John Lakos. *Large-Scale C++ Software Design*. Reading, MA: Addison-Wesley, 1995.
- [32] Eric Lengyel. *Mathematics for 3D Game Programming and Computer Graphics*, Second Edition. Hingham, MA: Charles River Media, 2003.
- [33] Gary B. Little. *Inside the Apple //e*. Bowie, MD: Brady Communications Company, Inc., 1985. (Available online at <http://www.apple2scans.net/files/InsidetheIIe.pdf>.)
- [34] Tuoc V. Luong, James S. H. Lok, David J. Taylor and Kevin Driscoll. *Internationalization: Developing Software for Global Markets*. New York, NY: John Wiley & Sons, 1995.
- [35] Steve Maguire. *Writing Solid Code: Microsoft's Techniques for Developing Bug-Free C Programs*. Bellevue, WA: Microsoft Press, 1993.
- [36] Scott Meyers. *Effective C++: 55 Specific Ways to Improve Your Programs and Designs*, Third Edition. Reading, MA: Addison-Wesley, 2005.
- [37] Scott Meyers. *More Effective C++: 35 New Ways to Improve Your Programs and Designs*. Reading, MA: Addison-Wesley, 1996.
- [38] Scott Meyers. *Effective STL: 50 Specific Ways to Improve Your Use of the Standard Template Library*. Reading, MA: Addison-Wesley, 2001.
- [39] Ian Millington. *Game Physics Engine Development*. San Francisco, CA: Morgan Kaufmann, 2007.
- [40] Hubert Nguyen (editor). *GPU Gems 3*. Reading, MA: Addison-Wesley, 2007.
- [41] Alan V. Oppenheim and Alan S. Willsky. *Signals and Systems*. Englewood Cliffs, NJ: Prentice-Hall, 1983.
- [42] Alan W. Paeth (editor). *Graphics Gems V*. San Francisco, CA: Morgan Kaufmann, 1995.
- [43] C. Michael Pilato, Ben Collins-Sussman and Brian W. Fitzpatrick. *Version Control with Subversion*, Second Edition. Sebastopol, CA: O'Reilly Media, 2008. (Commonly known as "The Subversion Book." Available online at <http://svnbook.red-bean.com>.)
- [44] Matt Pharr (editor). *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*. Reading, MA: Addison-Wesley, 2005.
- [45] Richard Stevens and Dave Raybould. *The Game Audio Tutorial: A Practical Guide to Sound and Music for Interactive Games*. Burlington, MA: Focal Press, 2011.
- [46] Bjarne Stroustrup. *The C++ Programming Language*, Special Edition (Third Edition). Reading, MA: Addison-Wesley, 2000.
- [47] Dante Treglia (editor). *Game Programming Gems 3*. Hingham, MA: Charles River Media, 2002.

- [48] Gino van den Bergen. *Collision Detection in Interactive 3D Environments*. San Francisco, CA: Morgan Kaufmann, 2003.
- [49] Alan Watt. *3D Computer Graphics*, Third Edition. Reading, MA: Addison Wesley, 1999.
- [50] James Whitehead II, Bryan McLemore and Matthew Orlando. *World of Warcraft Programming: A Guide and Reference for Creating WoW Addons*. New York, NY: John Wiley & Sons, 2008.
- [51] Richard Williams. *The Animator's Survival Kit*. London, UK: Faber & Faber, 2002.

Index

- #include, 148
- _DEBUG, 82
- __m128, *see* single instruction multiple data
- 2-blade, 373
- 2D sound, 955
- 3-blade, 374
- 3D Studio Max, 62, 669
- 3D sound, 955
- 80/20 rule, 99, 216, 317, 608
- A* algorithm, 1164
- AABB, *see* bounding box
- ABA problem, 297
- ABI, *see* application binary interface
- absorption
 - atmospheric, 918, 923, 956, 958, 964, 980
 - light, 633
 - sound, 917
- abstract factory, *see* design pattern
- AC-3, *see* audio, file formats
- accelerated processing unit, 227
- acoustic intensity, 915, 961
- acoustic pressure, *see* sound
- acoustical modeling, 956
- acoustics, 920
- ACP, *see* asset conditioning pipeline
- acquire fence, *see* memory ordering semantics
- acquire memory order, *see* memory ordering semantics
- action state machine
 - state layer, 787, 794
 - transition, 786
- ADC, *see* analog-to-digital conversion
- adding across a register, *see* single instruction multiple data
- additive blending, 769
- addressing mode, *see* CPU
- ADPCM, *see* pulse-code modulation
- Advanced Linux Sound Architecture, 994
- advanced vector extensions, 331
- affinity, *see* thread
- aggregation, 111, 1049, 1051, 1153
- AI, *see* artificial intelligence
- AIFF, *see* audio, file formats

- albedo map, 640
- algebra, 359
- algebraic simplification, *see* compiler, optimizations
- aliasing, 683, 950
- Alienbrain, *see* version control
- alignment, *see* memory
- allocation
 - chunky, 451, 514
 - delete operator (C++), 156
 - double-buffered, 435
 - dynamic, 155, 156, 426
 - heap memory, 156, 511
 - janitor, *see also* design pattern, 112
 - new operator (C++), 156
 - optimization, 426
 - pool, 430, 438, 512
 - resource chunk allocator, 514
 - single-frame allocator, 435
 - stack, 112
 - stack-based, 427, 438, 512
 - static, 155
- alpha, 623, 635
 - blending function, 676
 - testing, 676
- ALSA, *see* Advanced Linux Sound Architecture
- Altivec, *see* single instruction multiple data
- ALU, *see* arithmetic/logic unit
- ambient lighting, *see* lighting
- ambient occlusion, *see* lighting
- Ambisonics, 964
- amplifier, 926, 946, 982
- amplitude, 912, 933, 946, *see also* audio
- analog-to-digital conversion, 925, 948
- analytical geometry, 836
- anechoic chamber, 922
- angular frequency, 913, 933
- angular momentum, 872
- angular velocity, 867
- animation, 52, *see also* action state machine
 - additive blending, 769
 - attach point, 800, 808
 - blending, 755, 763
 - camera, 747
 - cel, 722
 - channel, 744, 747
 - clip, 734
 - constraint, 806, 816
 - cross-fade, 758, 759
 - difference clip, 769
 - ease curve, 759
 - Endorphin, 43
 - Euphoria, 43
 - event trigger, 746
 - film, 734, 736
 - flat weighted average, 788
 - floating-point channel, 747
 - frame, 722, 737
 - global timeline, 739
 - Granny, 42, 507, 516, 783, 790
 - hand-drawn, 722
 - Havok Animation, 42
 - idle, 722
 - instancing, 754
 - interobject registration, 808
 - joint, *see* joint
 - look-at, 816
 - looping, 722, 734, 738
 - metachannel, 746
 - morph target, 724
 - motion capture, 6
 - OrbisAnim library, 42
 - particle, 711
 - per-vertex, 724
 - phase, 738
 - playback rate, 739
 - post-processing, 774
 - procedural, 775
 - retargeting, 748
 - rigid, 723
 - run cycle, 734
 - sample, 737
 - skeletal, 62
 - socket, *see* animation, attach point
 - synchronization, 742
 - texture, 723
 - time scaling, 736, 739
 - traditional, 722
 - transition, 758, 759
 - updating, 1089
 - walk cycle, 734

- animation compression, 726, 777
 - channel omission, 778
 - curve-based, 783
 - key omission, 782
 - quantization, 778
 - sampling frequency, 782
 - wavelet compression, 783
- anisotropic, *see* texture, filtering
- ANSI, 462
- anti-commutative, 372
- antialiasing, 48, 683
- API, *see* application programming interface
- application binary interface, 176
- application programming interface, 40, 43, 975
- approximation
 - piecewise-linear, 746, 970
- architecture
 - runtime, 38
- archive file, 505
- area light, *see* lighting
- Argand plane, 934
- argument (complex), 934
- arithmetic/logic unit, 166
- array of structs, 1060
- articulatory speech synthesizer, 996
- artificial intelligence, 58, 538, 597, 822, 849, 901, 997, 1000, 1004, 1029, 1034, 1044, 1059, 1062, 1066, 1105, 1131, 1164
- ASM, *see* action state machine
- assembly language, 105
- assertion, 44, 86, 122, 125–127, 325
 - compile-time, 128, 129
 - lock-not-needed, 325, 1102
 - static, 128, 129
- asset conditioning pipeline, 59, 61, 499, 501, 668, 671, 1036
- associative, *see also* cache
 - array, 1139
 - property, 933
- asymmetric multiprocessing, 228
- asynchronous, *see* multitasking
- atomic, 262, 264, 267, 289
 - instruction, 292
 - operation, 262, 276, 549, 555
 - variable, 314
- ATRAC, *see* audio, file formats
- attach point, *see* animation
- attenuation
 - distance-based, 956, 957
- audible frequency band, 917
- audio, *see also* sound, 54
 - amplitude panning, 959
 - analog, 942
 - beating, 919
 - decay, 921
 - density, 921
 - diffuse tail, 920
 - diffusion, 921
 - ducking, 989
 - early reflections, 920
 - exclusion, 968
 - file formats, 951, 952
 - group, 989
 - instance limiting, 990
 - jack, 947
 - late reverberations, 920
 - mixing, 956, 962, 975, 980, 981, 988
 - obstruction, 968
 - occlusion, 968
 - pan, 956, 957, 975
 - constant gain, 961
 - constant power, 961
 - focus, 963, 964
 - pan pot, 980
 - perceived loudness, 914
 - perception of position, 923
 - pre-delay, 921
 - processing graph, 976
 - rendering, 54, 911, 955
 - spatialization, 956
 - updating, 1089
 - voice, 977
 - voice stealing, 991
- audio engine, 975
 - Quake, 54
 - Scream, 54
 - Unreal, 54
 - X3DAudio, 993
 - XACT, 993
 - XAudio2, 54, 993
- augmented reality, 27

- auxiliary send, 979
- average
 - weighted, 765
- AVX, *see* advanced vector extensions
- axis+angle rotation, 404
- azimuthal angle, 959

- band-limited, 948, 949
- bandwidth, *see* pipeline
- bank, *see* sound, bank
- barrier
 - compiler, 303
 - memory, 310
- barycentric coordinates, 766
- base, *see also* numeric base
- baseline, 717
- batched updating, 1090
- battle royale, *see* genre
- beating, *see* audio
- bel, *see* decibel
- Bézier triangle, 624
- “big O” notation, 445
- big-endian, *see* endian
- bilinear, *see* texture, filtering
- billboard, 49, 711, 714
- binary, *see* numeric base, 131
- binary heap, 442
- binary search tree, 442
- binary semaphore, 276
- binary space partitioning tree, 1018, *see* tree
- Bink Video, 1162
- biomechanical models, 43
- bit depth, 949
- bitwise operator, 176, 346
- bivector, 373
- bleach bypass, 48
- blend tree, 788, 792–796, 798, 801, 806
- blending, *see also* animation
 - audio, 957, 971
 - stage (graphics), 676
- Blinn-Phong lighting model, *see* lighting
- blocking algorithm, 289
- blocking function, 245–247, 251, 268, 270, 273, 289, 1102, 1110, 1111
- bloom, *see* lighting

- Blueprints, *see* Unreal Engine 4
- Bluetooth, 562
- Bode plot, 939
- Boost library, 40, 45, 253, 254, 447–449, 1081
- bounding box
 - axis-aligned, 411, 831
 - oriented, 411, 832
- bounding sphere tree, *see* tree
- bounding volume, 688
- branch dependency, 218
- branch penalty, 218
- branch prediction, 218, 219
- brawler, *see* genre
- BRDF, *see* lighting
- breakpoint, *see* debugger
- brush geometry, 62, 1018
- BSP tree, *see* tree
- BSS segment, *see* executable file
- BSSRDF, *see* lighting
- bubble-up effect, 1050
- build configuration
 - debug build, 86
 - development build, 86
 - hybrid build, 86
 - release build, 82
 - ship build, 86
- build rules, 502
- Bullet, 824
- bullet through paper, *see* tunneling
- bump mapping, *see* rendering
- bus
 - address, 174
 - audio, 947, 978
 - analog, 983
 - digital, 983
 - implementation, 983
 - latency, 984
 - master output, 982
 - preset, 990
 - voice, 978
 - data, 174
- busy-wait, 245, 251, 270, 273, 289, 294, 298, 328, 555
- byte, 132
- byte code, 1135, 1145

- C standard library, 43, 89, 412, 465, 486–488, 539, 589
- C++, 4
 - best practices, 105
 - bitwise operator, 176, 346
 - class, 106
 - declaration, 146–148, 150
 - definition, 146, 148
 - delete, 156
 - header file, 79
 - inheritance, 106
 - initialization order, 418
 - managed, 489
 - multiple inheritance, 107
 - new, 156
 - object model, 1022
 - postincrement, 445
 - preincrement, 445
 - preprocessor, 79
 - private, 149
 - public, 149
 - source file, 79
 - standard library, 40, 89, 114, 269, 448
 - standardization, 113
 - static, 149, 157
 - translation unit, 79, 144
 - user-defined literals, 459
 - versions of, 113
 - virtual inheritance, 107
 - volatile, *see* volatile
- C4 Engine, *see* Tombstone Engine
- C#, 34, 531, 1063
 - delegate, 1115
 - reflection, 1064, 1136
- cache, 191
 - coherency, 197, 630, 1092
 - coherency domain, 308
 - coherency protocol, 307
 - copy-back, 196
 - direct-mapped, 195
 - eviction, 195
 - friendly design, 1060
 - hit, 191
 - hit rate, 196
 - line, 192
 - MESI protocol, 307
 - miss, 191, 198
 - MOESI protocol, 307
 - multilevel, 196
 - replacement policy, 196
 - set associativity, 195
 - write-back, 196
 - write-through, 196
- call stack, 92, 153, 610
 - stack frame, 153
- callback function, 489, 492, 530
- camera, 47, 622, 655, 1163
 - debug, 605
 - imaging rectangle, 655
- Camtasia, 607
- Cartesian coordinates, 360
- Castle Wolfenstein 3D, 31, 625
- caustics, *see* lighting
- CCD, *see* continuous collision detection
- CD, *see* compact disc
- cel animation, 722
- cell broadband engine, 228
- center of mass, 857
- central arbitor, 285
- central processing unit, *see* CPU
- Cg, 672
- CgFX, 682
- Chain of Responsibility, 1122
- Chandra-Misra, 285
- change of basis, 388
- channel, *see* animation
- character dialog, 997
- character set
 - ANSI, 462
 - Unicode, 462
- cheats, 606
- circular wait, 282
- class, 106, 156
 - aggregation, 1049, 1051
 - composition, 1049, 1051
 - constructor, 520
 - coupling, *see* coupling
 - diagram, 107
 - hierarchy, 1046
 - bubble-up effect, 1050
 - monolithic, 1046
 - Unreal, 1046
- instance, 106, 156

- memory layout, 158
- mix-in, 107, 1049
- packing, 159
- pure virtual function, 163
- reflection, 1064
- scripted, 1153
- virtual function, 162
- classical mechanics, 854
- ClearCase, *see* version control
- clip, *see* animation, sound
- clipping, 411, 659, 674
- clipping plane, 46
- clock
 - global, 741
 - local, 741
- closed hash table, *see* container
- cloth
 - rendering, 649, 670
 - simulation, 674, 818, 909
- cloud computing, 225
- clouds, 714
- CLR, *see* Common Language Runtime
- CM, *see* center of mass
- code reuse, 38
- code segment, *see* executable file
- codec, 979
 - video, 1162
- coding standards, 118
- coefficient of restitution, 877
- Coffman conditions, 282
- COLLADA, 682
- collection, *see* container
- collinear vectors, 369
- collision, *see also* hashing
 - contact, 830, 849
 - detection, 42, 51, 52, 207, 359, 384, 411, 527, 595, 823, 825
 - shape, 829
 - sweep and prune algorithm, 847
 - updating, 1089
- color, 622, 633
 - channel, 634
 - log-LUV, 634, 702
 - model, 634
 - RGB, 634
 - space, 634
 - spectral, 633
- colorization, 719
- command line arguments, 473, 477
- Common Language Runtime, 34, 489
- commutative property, 368, 372, 933
- compact disc, 948
- compare-and-swap instruction, 295
- compile-time assertion, *see* assertion
- compiled language, 1135
- compiler, 78
 - build configurations, 81
 - debugging information, 83
 - GNU compiler, 83
 - optimizations, 81, 84, *see also* linker
 - algebraic simplification, 85
 - code inlining, 85
 - compile-time vs. link-time, 85
 - constant folding, 85
 - constant propagation, 85
 - dead code elimination, 85
 - instruction reordering, 85
 - local vs. global, 84
 - loop unrolling, 85
 - operator strength reduction, 85
 - peep-hole, 84
 - profile-guided, 85
 - optimizing, 316
 - project configuration tutorial, 88
 - project files, 80
 - solution files, 80
 - unresolved symbol error, 144
 - warnings, 81
- compiler barrier, *see* barrier
- compiler intrinsic, 294, 295, 333
- complex exponential, 934
- complex instruction set, 223
- complex number, 934
 - multiplication, 935
 - rotation, 935
- composition, 111, 1049, 1051, 1052, 1153
- compression, *see also* animation
 - compression
 - lossless, 951
 - quantization, 564, 778, 948, 949
 - texture, 642
- compressor, 983
- compute shader, 348, 350, 351, 672
- concurrency, 204, 205, 256, 544

- message passing, 257
- monitor, 288
- progress, 289
- shared memory, 204, 257
- condition variable, 273, 278, 548, 555
- configuration file, 471
- configuration space, 776
- conical sound source, 918
- consensus problem, 300
- conservative advancement, 845
- consistency model, *see* data-centric consistency model
- console
 - gaming platform, 8
 - in-game, 50, 604
- constant folding, *see* compiler, optimizations
- constant power pan law, *see* audio
- constant propagation, *see* compiler, optimizations
- constant register, *see* GPU
- constraint, 407, 806, 816, 854, 879, 883
- consume memory order, *see* memory ordering semantics
- contact, *see* collision
- container, 40, 441
 - array, 441
 - binary heap, 442
 - binary search tree, 442, 452
 - building custom, 447
 - deque, 442
 - dictionary, 442, 452, 508, 1008, 1065, 1141
 - dynamic array, 442, 451
 - graph, 443
 - hash table, 442, 452
 - open & closed, 452
 - list, 41, 442, 1141
 - map, 442
 - priority queue, 442
 - queue, 442
 - set, 443
 - stack, 442
 - tree, 442
 - vector (STL), 41, 442
- contention, 287
- context switch, 553
- continuity
 - C0, C1, C2, 758
 - motion, 758
- continuous collision detection, 845
- continuous-time signal, *see* signal
- control dependency, 218
- controller, *see* human interface device
- convergence, 863
- conversation, 1002
 - action, 1009
 - branching, 1004
 - context-sensitive, 1009
 - criterion, 1008
 - exclusivity, 1004
 - rule, 1008
 - speaker and listener, 1007
- convex polyhedron, 412
- convolution, 719, 930, 932, 933
- cooperative multitasking, 1140, 1154
- coordinate system, 384
 - Cartesian, 360
 - cylindrical, 360
 - hierarchy, 388
 - homogeneous clip space, 659
 - left-handed, 361
 - light space, 655, 703, 704
 - model space, 385, 630, 751
 - right-handed, 361
 - screen space, 663, 716
 - spherical, 360
 - tangent space, 635
 - texture space, 641
 - view space, 387, 656
 - world space, 386, 631
- copy on write, 457
- copy-back cache, 196
- core, *see* CPU
- coroutine, 204, 554, 1140
- coupling, 38, 46, 53, 66, 144, 1020, 1052, 1095
- CPU, 166
 - addressing mode, 178, 180
 - core, 226, 353
 - CPU-dependent game, 536
 - micro-operations, 172
 - out-of-order execution, 171, 217, 222, 224, 225, 239, 291,

- 300–302, 304, 309, 311, 315, 355
 - pipelined, 171, *see also* pipeline, 211, 213, 217, 221, 225, 445
 - register, 153
 - speculative execution, 239
 - stage, 212, *see also* GPU, stage
 - superscalar, 211, 221
 - utilization, 1105
 - very long instruction word, 211, 224
- crash report, 594
- critical
 - operation, 262, 264, 267, 289, 302, 303, 325, 327
 - race, 258
 - section, 270
- cross product, *see* vector
- cross-fade, 758, 759, 786, 1001
- crowd modeling, 996
- Crystal Space, 36
- CSV file, 467, 614
- cube map, 700
- cue, *see* sound
- culling, 15, 47
 - antiportal, 689
 - back-face, 627
 - frustum, 688, *see* frustum
 - occlusion, 15, 18, 47, 688
 - portal, 15, 689
 - potentially visible set, 688
 - visibility determination, 688
- curve, 759
- cvar, 474
- CVS, *see* version control
- cylindrical coordinates, 360

- D-cache, *see* data cache
- DAC, *see* digital-to-analog conversion
- DAG, *see* directed acyclic graph
- damping, 860
- dashpot, *see* damping
- data cache, 196
- data definition language, 1135
- data dependency, *see* dependencies
- data parallelism, *see* parallelism
- data pathway, 1132

- data race, 205, 259, 281
- data segment, *see* executable file
- data type, 138, 146
- data-centric consistency model, 266
- data-driven, 12, 1024
 - cost of, 1024
- DCC, *see* digital content creation app
- dead code elimination, *see* compiler, optimizations
- deadlock, 275, 281
- debug build, *see* build configuration
- debug drawing, 50, 595
- debugger, 50, *see also* call stack
 - breakpoint, 92, 95–97
 - debugging optimized builds, 97
 - single-step, 92
 - watch window, 93
- decal, 48, 712
- decay, *see* audio
- decibel, 914, 915
- decimal, *see* numeric base
- declaration, *see* C++
- declarative language, 1136
- decode, *see* CPU, stage
- decomposition, *see* task decomposition
- deep copy, 1127
- deferred rendering, 709
- definition, *see* C++
- deformable body, 825, 903, 909
- degree of parallelism, 1105
- degrees of freedom, 406, 856, 883
- Delaunay triangulation, 766
- delay slot, 215, 225
- delegate (C#), 1115
- delete, *see* C++
- demodulation, 951
- denormalized, *see* floating-point
- density, *see* audio
- dependencies, 214, 222
 - cycle, 38
 - data, 215
 - engine subsystems, 1093
 - game objects, 1093
 - thread, *see* thread
- dependency graph, 1105
- depth buffer, 664, 704
 - shadow mapping, 704

- testing, 676, 716
- z-testing, 675, 692, 693
- depth of field blur, 719
- deque, 442
- derivative of a vector, 858
- design pattern, 111
 - abstract factory, 111, 1042
 - Chain of Responsibility, 1122
 - command, 1117
 - factory, 1065
 - iterator, 111, 118, 430, 443, 445, 1149
 - janitor, 111, 112, 320, 327
 - RAII, 111, 125, 320, 327
 - singleton, 111, 418
- destructible objects, 903, 1018
- development build, *see* build
 - configuration
- development kit, 461, 616
- device driver, 975
- dialog action, 1009
- dialog system, 997
- diamond inheritance problem, 1049
- dictionary, *see* container
- difference clip, 769
- diffraction, *see* lighting, 917, 970
- diffuse
 - lighting, *see* lighting
 - tail, 920
 - texture map, 640
- diffusion, *see* audio
- digital content creation app, 59, 62, 501, 669
- Digital Molecular Matter, 825
- digital signal processing, *see* signal processing
- digital-to-analog conversion, 951
- dining philosophers, 285
- direct lighting, *see* lighting
- direct memory access controller, 984
- directed acyclic graph, 61, 443, 1109
- directed graph, 1109
- direction vector, 365
- directional light, *see* lighting
- directional sound source, 918
- DirectX, 41, 46, 91
 - perspective projection matrix, 660
 - view space, 387
- Dirichlet conditions, 938
- disassembly, 98
- discrete-time signal, *see* signal
- disk operating systems, 233
- dispatching, 224
- displacement mapping, *see* rendering
- display device, 622
- distance field, *see* signed distance field
- distance-based attenuation, *see* attenuation
- distributed computing, 229
- distributed shared memory, 257
- distributive property, 368, 372, 933
- division by w , 381
- DivX, 1162
- DMAC, *see* direct memory access controller
- DMM, *see* Digital Molecular Matter
- DOC, *see* degree of parallelism
- DOF, *see* degrees of freedom, depth of field
- Dolby Digital, *see* audio, file formats
- Doom, 11, 31, 625, 1025
- DOP, *see* degree of parallelism
- Doppler effect, 922, 956, 957, 973
- dot product, *see* vector
- double dispatch, 842
- DRC, *see* dynamic range compression
- dry sound, *see* sound
- DSP, *see* signal processing
- DTS, *see* audio, file formats, *see* surround sound
- duality, 940
- duck typing, 1141
- ducking, *see* audio
- dynamic allocation, *see* allocation
- dynamic array, 442
- dynamic link library, 80
- dynamic range compression, 983
- dynamics (physics), 854
- early reflections, 920
- ease curve, 759
- echo, 920
- Edge, *see* PlayStation Edge library
- effect file, *see* shader

- effective sound pressure, *see* sound
- ELF, *see* executable
- elision, 115
- emissive, *see* lighting
- encapsulation, 106
- endian, 140
 - big, 140
 - little, 140, 334
 - multibyte quantity, 140
 - swapping, 141
- Endorphin, 43
- energy, 875
- engine, 11
 - line between engine and game, 11
 - reusability gamut, 12
- engine configuration, 470
 - OGRE, 475
 - Quake, 474
 - Uncharted/TLOU, 475
- environment map, 48, 700, 706
- environment variable, 236, 473
- EQ, *see* equalizer
- equal-loudness contour, 916
- equalizer, 940, 941, 982
- equilibrium, 882
- error, *see also* compiler
 - conditions, 119
 - numerical methods, 863
 - quantization, 949
- Euler angles, 386, 403
- Euler's formula, 936, 937
- Euphoria, 43
- event, 57, 531, 1040, 1114
 - animation trigger, 746
 - arguments, 1118
 - data-driven, 1131
 - debugging, 1128
 - driven, 37, 57, 1114, 1122
 - forwarding, 1116, 1122
 - GUI-based, 1133
 - handling, 1120, 1151
 - memory allocation, 1128
 - object, 279
 - priorities, 1126
 - queuing, 1124
 - registering interest, 1123
 - sending, 1152
 - trigger, 746
 - types, 1117
- evict, *see* cache
- exception handling, 123
 - exception object, 123
- exchange instruction, 295
- exclusion, *see* audio
- executable
 - BSS segment, 152
 - code segment, 151
 - data segment, 151
 - ELF format, 151
 - file, 79, 151
 - read-only data segment, 152
 - rodata segment, 152
 - segment, 151
 - text segment, 151
- explicit parallelism, 206, 225
- explosions, 902
- exponent, *see* floating-point
- expression tree, 792
- F#, 1136
- facing, 369
- fact dictionary, 1008
- factory, *see* design pattern
- fall-off, *see* sound, pressure
- fan, *see* triangle
- fast Fourier transform, 939
- feedback, 927
- fence, *see also* barrier, *see* memory
 - ordering semantics
- fetch, *see* CPU, stage
- FFT, *see* fast Fourier transform
- fiber, 204, 554
- field of view, 46
- fighting game, *see* genre
- file I/O
 - asynchronous, 489, 1072
 - buffered, 486
 - deadline, 492
 - synchronous, 488
- file scope, *see* scope
- file system, 482
 - path, 458, 482, 484
 - search path, 485
 - wrapping, 487

- filter, 940, 979
 - comb, 920
 - low-pass, 940
 - notch, 940
 - passband, 940
 - post-send, 980
 - pre-send, 980
 - stopband, 940
 - texture, *see* texture, filtering
- finite state machine, 786, 1042, 1087, 1154, 1157
- first-person shooter, *see* genre
- fixed-function pipeline, 672
- fixed-point, *see* integer, 133
- flashlight, *see* lighting
- flat weighted average, *see* animation
- floating-point, 133
 - denormalized, 136
 - exponent, 134
 - infinity, 135
 - machine epsilon, 136
 - magnitude, 134
 - mantissa, 134
 - not-a-number, 135
 - precision, 134
 - significant digits, 134, 135
 - subnormal, 136
 - unit in the last place, 137
- floating-point unit, 166
- flow control, 1157
- FLT_MAX, 135
- fluid dynamics, 715, 910, 924
- fluid simulation
 - position based, 910
- flushed, 218
- Flynn's taxonomy, 207
- FMOD Studio, 995
- FMV, *see* full-motion video
- FName, 459
- FO min and max, *see* sound, fall-off
- focus, *see* audio
- Folly library, 40, 45, 450
- font rendering, *see* text rendering
- foot sliding, 813, 815
- force, 373, 859
 - as a function, 860
- forward kinematics, 775
- Fourier transform, 933, 938, 940
- FPS, *see* genre, *see* frame rate
- FPU, *see* floating-point unit
- fragment, 664
- frame, *see* animation
- frame buffer, 663
 - back buffer, 538
 - double buffering, 663
 - front buffer, 538
 - swapping, 538
 - triple buffering, 664
- frame rate, 10, 534, 736
 - governing, 537, 538
- frame-to-frame coherency, *see* temporal coherency
- framework, 529
- Fraps, 607
- free store, *see also* allocation, 156
- frequency, 913
 - angular, 913
 - domain, 933, 938
 - fundamental, 938
 - response, 920, 940
- friction, 880
- frustum, 47
- FSM, *see* finite state machine
- full fence, 310, *see* memory ordering semantics
- full-motion video, 49, 735, 1019, 1161
- function
 - signature, 146
- functional language, *see* language
- Fusion, 37
- futex, 271
- Fx Composer, 670
- G-buffer, 709
- G-factor, 1015
- gain, 926, 946, 979, 983
- game
 - definition of, 8
- game controller, *see* human interface device
- game development team
 - artist, 6
 - composer, 6
 - engineer, 5

- game designer, 7
- producer, 7
- publisher, 8
- sound designer, 6
- voice actor, 6
- writer, 7
- game engine, *see* engine
- game flow control, 1157
- game loop, 526, 544, 899
 - parallelizing, 545
 - pausing, 605
 - Pong*, 527
 - single-stepping, 532, 605
 - sleep, 537
 - slow motion, 605
- game object, 1021
 - asynchronous updating, 1102
 - attribute, 1021
 - batched updating, 1090
 - behavior, 1021, 1041
 - dependencies, 1093
 - instance, 1021
 - linkage to engine, 1041
 - persistence, 1042
 - queries, 1042
 - query, 1085
 - reference, 1150
 - references, 1042, 1079
 - spawning, 1040
 - state, 1087, 1100
 - state caching, 1100
 - time-stamping, 1101
 - type, 1021
 - types, 1041
 - unique id, 458, 1041, 1079, 1086
 - updating, 527, 1040, 1086, 1093, 1095
- game object model, *see* object model
- game state, 1016
- game world, 9, 56, 64, 1016, 1018
 - chunk, 1019, 1029, 1062
 - editor, 59, 64, 714, 1021, 1023, 1025, 1030–1033, 1035, 1043
 - Hammer, 64, 1025
 - Radiant, 64, 1025
 - Sandbox, 1025
 - UnrealEd, 64, 66, 1025
- gameplay, 1015
 - flow, 1015, 1019, 1040
 - foundation system, 55, 1039
 - hub, 1019
 - linear, 1019
 - objectives, 1015, 1019, 1040
 - open world, 1019
 - player mechanics, 55, 1015, 1162
 - region, 1033
 - task, 1019
 - water, 715
- GameSalad, 37
- gamma
 - correction, 718
 - response curve, 718
- gamut, *see* lighting
- garbage collection, 1081
- Gaussian elimination, 378
- gcc, *see* GNU C/C++ compiler
- general-purpose GPU programming, *see* GPGPU
- genre, 13
 - battle royale, 24
 - brawler, 19
 - fighting, 17
 - first-person shooter, 14
 - massively multiplayer online
 - game, 9, 13, 23
 - platformer, 15
 - racing, 19
 - real-time strategy, 21
 - simulation, 31
 - strategy, 21
 - turn-based strategy, 21
- geometry, *see also* mesh
 - brush, *see* brush geometry
- gimbal lock, 403, 404, 871
- git, *see* version control
- GJK algorithm, 839
- Glide, 41
- global illumination model, *see* lighting
- global namespace, 118
- global optimizations, *see* compiler, optimizations
- globally unique identifier, 458, 507, 517
- gloss map, 699
- glyph atlas, 716

- GNU C/C++ compiler, 78, 336
- Gouraud shading, 637
- GPGPU, 41, 348, 672
- GPU, 672
 - command list, 692
 - compute unit, 353, 354
 - kernel, 350
 - lock step execution, 355
 - register, 678
 - SIMD unit, 350, 354
 - stage, 355
 - thread, 354, 355
 - thread block, 353
 - thread group, 353, 354
 - warp, 355
 - wavefront, 355
- grammar, 792
- Granny, 42, 507, 516, 783, 790
- graphical shading language, 670
- graphical user interface, 49
- graphics processing unit, *see* GPU
- gravity, 856, 875, 888
- grenade physics, 902
- group, *see* audio
- GUI, *see* graphical user interface
- GUID, *see* globally unique identifier

- H.26x, 1162
- Hadamard product, 364, 676
- hair, 647, 674, 710, 909
- Half-Life 2, 32, 33
- Hammer, *see* game world editor
- handle, 440, 554, 1042, 1079, 1082
 - sound, 987
 - stale, 1083
- hardware T&L, 41, 672
- harmonic, 938
 - scaling in time domain, 974
- hashing, *see also* container, 453
 - chaining, 453
 - collision, 452, 455, 459
 - linear probing, 455
 - open addressing, 453
 - quadratic probing, 455
 - Robin Hood hashing, 456
- Haskell, 1136
- Havok, 42, 824
 - Havok Animation, 42
- HDMI, 954
- head-related transfer function, 924
- header file, *see* C++
- headphones, 944
- headroom, 962
- heads-up display, 49
- heap memory, 156
- height field, *see* terrain
- heightmap, *see* texture
- HeroEngine, 35
- Hertz, 534
- heterogeneous system architecture, 678
- hex editor, 102
- hexadecimal, *see* numeric base, 132
- HID, *see* human interface device
- hierarchy, *see also* coordinate system, *see also* joint
- high dynamic range, *see* lighting
- high water mark, 451, 616
- hit rate, *see* cache
- hold and wait, 282
- homogeneous coordinates, 379
- horizontal add, *see* single instruction
 - multiple data
- Houdini, 64
- HRTE, *see* head-related transfer function
- HSA, *see* heterogeneous system
 - architecture
- HTML, 1136
- HUD, *see* heads-up display
- hUMA, *see* heterogeneous system
 - architecture
- human interface device, 53, 559
 - abstract control indices, 584
 - accelerometer, 566, 567
 - actuator, 570
 - analog axis, 564
 - analog input, 564
 - and game loop, 527
 - audio, 570
 - button, 563, 564
 - chord, 575
 - context-sensitive controls, 585
 - control mapping, 584
 - cross-platform, 582
 - dead zone, 571

- disabling, 586
- DualShock, 562, 566, 567
- force-feedback, 53, 570
- gesture, 577
- infrared sensor, 567
- input event, 574
- interrupt, 562
- keyboard, 54
- mouse, 566
- multiplayer, 582
- PlayStation Eye, 568
- polling, 561
- relative axis, 566
- rumble, 569
- sequence, 577
- signal filtering, 572
- system requirements, 571
- Wiimote, 559, 566, 567
- XInput, 562, 563
- hybrid build, *see* build configuration
- Hydro Thunder*, 1044
- hyperthreading, 225
- hysteresis, 971
- I-cache, *see* instruction cache
- I-Collide, 52
- IDE, *see* integrated development environment
- IGC, *see* in-game cinematic
- IID, *see* interaural intensity difference
- IK, *see* inverse kinematics
- image
 - bitmapped, 634
 - bits per pixel, 634
 - of program, 151
 - sampling, 683
- image-based lighting, *see* lighting
- imaginary number, 934
- imperative language, 1136
- implicit parallelism, 206, 211
- impulse
 - physics, 876, 877
 - response, 928, 932, 966
 - unit, *see* unit impulse
- impulse response, 931
- in-game cinematic, 49, 735, 991, 1019
- independent variable, 924
- index buffer, 628, 630
- indirect lighting, *see* lighting
- inertia tensor, 870
- inheritance, 106, 162, 1022, 1067, 1153
 - deadly diamond, 107
 - diamond problem, 1049
 - multiple, 107, 162, 1049
 - virtual, 107
- initialization order (C++), 418
- inline function, 83, *see* compiler, optimizations, 148
- inner product, 367
- input register, *see* GPU
- instance limiting, *see* audio
- instanced geometry, 1018
- instancing, *see* animation
- instantaneous acoustic pressure, *see* sound
- instruction cache, 196
- instruction reordering, *see* compiler, optimizations
- instruction stream, 354
- instruction-level parallelism, 214
- Insure++, 50, 102
- integer, 132
 - fixed-point, 133
 - sign and magnitude, 132
 - sign bit, 132
 - signed, 132
 - two's complement, 132
 - unsigned, 132
- integrated development environment, 78
- integration
 - explicit Euler, 366, 535
 - numerical, 10, 535
- intensity, *see* lighting
- inter-object communication, *see* event system
- interaural
 - intensity difference, 959
 - time difference, 923
- interconnect bus, 308
- interface, 118
- interference, 919, 944
 - constructive and destructive, 919
- interpolation

- linear, 375, 400, 546, 755
- spherical linear interpolation, 401
- temporal, 757
- vertex attributes, 635
- interpreted language, 1135
- interrupt, 95, 186, 232, 234, 247, 292
 - service routine, 177, 562
- interruption
 - of character dialog, 1001, 1003, 1004
 - of critical operation, 261, 264, 291
- intersection, *see also* collision, 829
 - AABB versus AABB, 838
 - point versus sphere, 836
 - sphere versus sphere, 836
- intrinsic, *see* compiler intrinsic
- inverse kinematics, 775, 811
- invocation, 262
- Irrlicht, 36
- ISR, *see* interrupt, service routine
- ITD, *see* interaural time difference
- iterator, *see* design pattern
- janitor, *see* design pattern
- Java, 1063
- job, 549
 - counter, 554
 - declaration, 549
 - kicking, 549
 - synchronization, 555
 - system, 545, 549, 1102
- join, *see* thread
- joint, *see also* animation
 - coordinate, 730
 - index, 728
 - name, 728
 - nonuniform scale, 731
 - parent, 728
 - root, 728
 - scale, 731
- joystick, *see* human interface device
- kd-tree, 47, 695
- kernel, *see* GPU
- kernel call, 267
- kerning, 717
- key frame, 736
- key-value pair, 452, 508, 1031, 1032, 1065, 1119, 1139
- keyboard, *see* human interface device
- kicking a job, *see* job
- Killzone 2, 709
- kinematics, 785
 - end effector, 775
 - forward, 775
 - inverse, 775, 811
- Kismet, *see* Unreal Engine 4, Blueprints
- Kynapse, *see* artificial intelligence
- L1 cache, *see* cache
- lambda, 1144
- lane, *see* single instruction multiple data
- language, 106
 - compiled, 1135
 - declarative, 1136
 - functional, 1101, 1136
 - imperative, 1136
 - interpreted, 1135
 - object-oriented, 106, 1136
 - procedural, 1136
 - reflection, 1022, 1042, 1052, 1064, 1065, 1136
- late function binding, 1115
- late reverberations, 920
- latency
 - audio bus, 984
 - memory access, 188
 - pipeline, 214
- LeadWerks Engine, 35
- left-hand rule, 371
- level of detail, 625, 714, 715
- lexically scoped, *see* scope
- LFE, *see* low-frequency effects
- libgcm, 41
- library, 79, 529
- lifetime
 - debug primitive, 597
 - resource, 503
- light map, 48, 653, 672
- lighting, 633, 647
 - absorption, 633
 - ambient, 649, 654
 - ambient occlusion, 705
 - area light, 655

- Blinn-Phong lighting model, 652
- bloom, 48, 655, 702
- BRDF, 652
- BSSRDF, 707
- caustics, 706
- diffraction, 633
- diffuse, 649
- direct, 647
- directional, 654
- emissive, 655
- flashlight, 655
- gamut, 633
- global illumination model, 648, 702
- high dynamic range, 634, 655, 702
- image-based, 698
- indirect, 648, 702
- intensity, 633
- interactions with matter, 633
- local illumination model, 647
- medium, 633
- per-pixel shading, 637
- Phong reflection model, 649
- point light, 654
- precomputed radiance transfer, 708
- radiosity, 648
- ray tracing, 648
- reflection, 633, 706
- refraction, 633
- scattering, 634, 707
- source, 622, 653
- spec map, 699
- specular, 649
- specular power map, 670, 699
- spot light, 654
- static, 653, 672
- transmission, 633
- transport model, 622, 647
- viewing direction, 650
- wavelength, 633
- Lightwave, 669
- line, 408
- line of sight, 595, 849, 1103
- line-level, 945
- linear, 926
 - algebra, 359
 - approximation, 625, 745
 - momentum, 859
 - probing, *see* hashing
 - time-invariant system, 926
 - velocity, 858
- linearizability, 266
- link register, 297
- linkage, 149
 - external, 149
 - internal, 149, 151, 153
- linker, 78
 - optimizations, 85
- Lisp, 477, 797, 1144
- list, *see* triangle, *see* container
- listener, 956
- little-endian, *see* endian
- livelock, 283
- load linked/store conditional, 297
- local illumination model, *see* lighting
- local optimizations, *see* compiler, optimizations
- local variable, *see* variable
- localization, 456, 462, 466
- location tag, 1009
- location-based entertainment, 30
- locator, 747
- lock-free algorithm, 267, 287, 290, 328, 1102
- lock-not-needed assertion, *see* assertion
- locomotion
 - cycle, 722
 - noise, 773
 - pivotal, 761
 - targeted, 761
- LOD, *see* level of detail
- logarithmic, 915
- logging, 589
 - channel, 592
 - to file, 593
 - verbosity, 591
- Loki library, 40, 450
- look-up table, 517, 521, 522, 640, 674
- loop unrolling, *see* compiler, optimizations
- looping, *see* animation
- lossless compression, *see* compression
- loudness, *see* audio

- low-frequency effects, 944, 959, 982
- low-pass filter, 572
- LPCM, *see* pulse-code modulation
- LTI system, *see* linear time-invariant system
- LU decomposition, 378
- Lua, 1139
- Lumberyard, 36
- LUT, *see* look-up table

- machine epsilon, *see* floating-point
- macro, 333
- Macromedia Fusion, 37
- madd instruction, 340
- magnitude, *see also* floating-point
 - complex, *see* complex number
 - vector, *see* vector
- make, 86
- managed C++, 489
- manager, 418
- manifest constants, 152
- mantissa, *see* floating-point
- manycore, *see also* GPU, 210
- mass, 857
- massively multiplayer online game, *see* genre
- material, 682
 - editor, 670
 - system, 46
 - visual, 646, 670
- math library, 44
- matrix, 375
 - 3×3 , 380, 404
 - 4×3 , 384
 - 4×4 , 343, 363, 376, 380, 382, 390, 430, 730, 756, 769
 - affine, 376
 - camera-to-world, 46
 - column matrix, 377
 - conversion to quaternion, 399
 - identity, 378
 - in-memory representation, 392
 - inverse, 378
 - isotropic, 376
 - joint-to-model, 752
 - model-to-world, 369, 390, 631, 754
 - orthographic projection, 662
 - orthonormal, 376
 - perspective projection matrix, 660
 - product, 376
 - pure rotation, 404
 - row matrix, 377
 - special orthogonal, 376
 - transpose, 379
 - view-to-world, 656
 - world-to-view, 656
- matrix palette, 753
- Maya, 62, 669
- mechanics
 - classical, 854
- medium, *see also* lighting
- MEL language, 502, 1134
- Meltdown exploit, 239
- memory
 - access cycle, 293
 - access patterns, 426
 - alignment, 159, 333, 431
 - cache, *see* cache
 - card, 570
 - controller, 160, 166, 176, 184, 212, 227, 291, 356
 - corruption, 101
 - debug memory, 461
 - defragmentation, 439, 1077
 - fence, 304, *see also* barrier, 316
 - fragmentation, 437
 - in-game statistics, 615
 - leak, 101, 615
 - management, 44, 426, 511
 - management unit, *see* memory, controller
 - ordering bugs, 291
 - ordering semantics, 305, 311, 315
 - acquire, 311, 312, 316
 - consume, 316
 - full fence, 311, 316
 - relaxed, 315
 - release, 311, 312, 316
 - relocation, 439, 987, 1077
 - shared, *see* concurrency
 - small memory allocator, 1076
 - stick, 487
 - virtual, 437, 615
- menu

- in-game, 50, 475, 601
- mesh, 61, 121, 625
 - constructing, 627
 - instance, 631, 1090
 - progressive, 626
 - static, 1018, 1029
 - submesh, 646
- MESI, 197
- MESIF, 197
- message, *see also* event
 - map, 1121
 - passing, *see* concurrency
 - pump, 46, 529
- metric, *see* SI units
- mic-level, 945
- micro-operations, 172
- microphone, 942
- Microsoft Excel, 467, 614
- Microsoft Visual Studio, 70, 78
- Miles Sound System, 995
- MIMD, *see* multiple instruction
 - multiple data
- Minkowski sum/difference, 839
- mipmapping, 643, 679
- MISD, *see* multiple instruction single
 - data
- mix snapshot, 990
- mix-in class, 1049
- mixed reality, 27
- mixing, *see* audio
- MKS system of units, 856
- MMO, *see* genre
- MMX, *see* streaming SIMD extensions
- mod community, 11
- model
 - 3D, *see* mesh
 - analytic, 10
 - closed-form, 10
 - mathematical, 9
 - numerical, 10
- modeler (3D), 6
- modulation, 940
- MOESI, 197
- moiré pattern, 643
- moment of inertia, 867
- momentum
 - angular, 872
 - linear, 859
- monitor, *see* concurrency
- MonoGame, 34
- monolithic class hierarchy, 1046
- motion blur, 719
- motion capture, 6
- movie capture, 607
- movie player, 1161
- moving average, 537, 572
- MP3, *see* audio, file formats
- MPEG, 1162, *see also* audio, file formats
- multibyte quantity, *see* endian
- multicore, 210, 226, 437
- multilevel cache, *see* cache
- multiplayer, 55
 - human interface devices, 582
 - networked, 55, 1162
 - replication, 1042
 - split-screen, 55
- multiple inheritance, 1049
- multiple instruction multiple data, 207, 348
- multiple instruction single data, 207
- multiplication as complex rotation, 935
- multiply defined symbol error, 146
- multitasking, 540, 545
 - asynchronous programming, 1102
 - cooperative, 234, 250, 554, 1140, 1154
 - GPU, 672
 - interfacing with game object
 - update, 1102
 - job, 545
 - preemptive, 40, 230, 234, 247, 250, 554, 1140
 - pthreads, 545
 - sleep, 492, 537, 546
 - SPURS, 545
 - thread, 487, 492, 545
- music, 1010
- mutex, 112, 245, 264, 267–269, 271, 273, 282, 292, 555
- mutual exclusion, *see* mutex
- namespace, 118
- natural number, 132
- NDEBUG, 82

- nearest neighbor, *see* texture, filtering
- negative reinforcement, 919
- new, *see* C++
- Newton's law of restitution, 876
- Newton's laws of motion, 859
- Newtonian mechanics, 854
- non-blocking algorithm, 289, 290
 - lock-free, 289
 - obstruction-free, 289
 - wait-free, 289
- non-blocking function, 268, *see also*
 - blocking function, 1102
- non-player character, 15, 56, 734, 742
- noninteractive sequence, 735
- nonuniform rational B-spline, 624
- normal, *see also* vector
 - map, *see* texture
 - of plane, 369
- Novodex, *see* PhysX
- NPC, *see* non-player character
- NTSC, 534, 542, 663
- numeric base
 - binary, 131
 - decimal, 131
 - hexadecimal, 132
- numerical methods, 863
- NURBS, *see* nonuniform rational B-spline
- Nyquist frequency, 949
- OBB, *see* bounding box
- object file, 79
- object model, *see also* game object model
 - C++, 1022
 - component, 1052, 1055
 - component creation, 1053
 - component ownership, 1053
 - Excel, 1022
 - game, 56, 1022, 1040
 - Hydro Thunder*, 1044
 - interface, 1022
 - multitasking, 1107
 - object-centric, 1043, 1044
 - OMT, 1022
 - property-centric, 1043, 1060
 - runtime, 1023, 1043
 - software, 1022
 - tool-side, 1023
 - unique ids, 1056
- object-oriented, *see* language
- obstruction, *see* audio
- obstruction-free algorithm, 290
- Ocaml, 1136
- occlusion, *see* audio
- octal, *see* numeric base
- octree, 47, 694
- ODE, *see* Open Dynamics Engine
- Ogg Vorbis, 953
- OGRE, 36, 45, 47, 91, 139, 140, 789
- one-frame-off bug, 1099
- OOO execution, *see* CPU
- opacity, 623
- open addressing, *see* hashing
- Open Dynamics Engine, 42, 823
- open hash table, *see* container
- open-source software, 34
- OpenAL, 994
- OpenGL, 41, 46
 - perspective projection matrix, 660
 - view space, 387
 - view volume, 659
- OpenTissue, 825
- operating system, 40, 43, 79, 80, 151–153, 156
 - DOS, 483
 - Mac OS, 483
 - Microsoft Windows, 483
 - UNIX, 483
- operator overloading, 445
- operator strength reduction, *see*
 - compiler, optimizations
- optical audio connector, 954
- optimization, *see also* compiler,
 - optimizations, 99, 165, 216, 317, 447, 608
- OrbisAnim library, *see* animation
- ordinary differential equation, 860, 869, 873
- orthogonal, 363
- OS, *see* operating system
- out-of-order execution, *see* CPU
- outer product, 367, 370
- output register, *see* GPU
- overdraw, 692

- overlay, 49, 657, 716, *see* rendering
 - graphical user interface, 32, 80, 111
 - heads-up display, 657
- package file, 497, 499, 506
- page fault, 186, 239
- painter's algorithm, 664
- PAL, 534, 542, 663
- pan, *see* audio
- Panda3D, 36, 1143
- parallax occlusion mapping, *see* rendering
- parallelism, 203, 205
 - data, 207, 348, 544, 546
 - explicit, 206, 225
 - implicit, 206, 211
 - task, 207, 544, 545
- parallelogram
 - area of, 371
- parametric equation, 408, 848
 - surface, 624
- Pareto principle, *see* 80/20 rule
- particle system, 46, 48, 64, 711, 1089
- passing
 - of reads and writes, 310
- patch, 624
 - Bézier, 624
 - bicubic, 624
 - N-patch, 624
 - nonuniform rational B-spline, 624
- path tracing, 970
- Pawn, 1141
- PCM, *see* pulse-code modulation
- peep-hole optimization, *see* compiler, optimizations
- penumbra, 655, 703
- per-pixel shading, *see* lighting
- per-user options, 474
- perceived loudness, *see* audio
- perception of position, *see* audio
- perceptual coding, 952
- Perforce, *see* version control
- periodic, 913, 938
- periphonic, 964
- perpendicular
 - axes, 360, 837
 - distance, 410, 658, 688
 - vectors, 363, 369, 370, 392, 635, 638
 - wave, 917
- persistence, 1042
- perspective, 657
- phantom image, 959
- phase, *see also* animation
 - of complex number, 934
 - shift, 913, 919
 - vocoder, 974
- Phong reflection model, *see* lighting
- photorealism, 622
- PhyreEngine, 34
- physically based sound synthesis, 996
- physics, *see also* collision, 51, 527, 674
 - and fun, 819
 - Havok, 52
 - library, 42
 - Havok, 42
 - Open Dynamics Engine, 42
 - PhysX, 42
 - Open Dynamics Engine, 52
 - PhysX, 52
 - rigid body dynamics, 51, 726, 854
 - simulation, 51
 - updating, 1089
 - water, 715
 - world, 828
- Physics Abstraction Layer, 825
- PhysX, 42, 824
- piecewise-linear approximation, 746, 970, *see* approximation
- pinnae, 924
- pipeline, 213, 667, 672, 976, 1092
 - asset conditioning, *see* asset conditioning pipeline
 - bandwidth, 214
 - latency, 214, 667
 - throughput, 214, 667
 - tools, *see* asset conditioning pipeline
- pitch, 386
- pixel, 622, 655
- placement new, 520
- plain old data structure, 114, 520
- plan view, 657
- plane, 369, 409, 658
 - distance to origin, 409

- point-normal form, 409, 658
- platform independence layer, 43
- platformer, *see* genre
- player I/O, *see* human interface device
- playlist, 1011
- PlayStation Edge library, 41, 42
- PlayStation Network, 481, 487
- POD, *see* plain old data structure
- point, 360
 - arithmetic, 365
- point light, *see* lighting
- pointer fix-up, 519
- polar pattern, 942
- polarization, 917
- polygon
 - rendering, 624, 625
 - soup, 854
- polyhedron
 - convex, 412
- polymorphism, 109, 162
- polyphony, 977
- Portable Network Graphics, 642
- portal, 47
 - culling, 689
 - sound, 972
- pose
 - as change of basis, 732
 - bind pose, 728, 729, 751
 - current, 729
 - current pose, 751
 - global, 733
 - in-memory representation, 732
 - interpolation, 736
 - local, 730
 - T-pose, 729
- position based fluid simulation, 910
- positive reinforcement, 919
- post effect, 48, 719
- post-load initialization, 521
- postincrement (C++), 445
- potentially visible set, 47
- potentiometer, 947
- power, 961
- power processing unit, *see* PPU
- pre-amp, 982
- pre-delay, *see* audio
- precision, *see* floating-point
- precomputed radiance transfer, *see* lighting
- predication, 221, 346
 - vector, 344, 346, 347
- preemption, *see* multitasking
- preincrement (C++), 445
- preprocessor, *see* C++
- pressure, *see also* sound, 912
- primitive
 - debug drawing, 597
 - geometric, 46, 646
 - mesh-material pair, 646
 - submission, 691
- primitive data type, 146
- printf debugging, 589
- priority, *see also* thread
 - dialog, 1001
 - inversion, 284
 - voice, 991
- private, *see* C++
- procedural language, 1136
- processor utilization, 545, 1105
- producer-consumer problem, 271
- profile trap, 557
- profile-guided optimizations, *see* compiler, optimizations
- profiling tools, 50, 99
 - hierarchical, 609
 - in-game, 608
 - instrumenting, 100, 611
 - statistical, 100
- program order, 262
- program stack, *see* call stack
- progress, *see* concurrency
- projection, 368, 657
 - orthographic, 21, 657, 662, 716, 1029
 - perspective, 22, 657, 660, 661, 1029
 - perspective foreshortening, 657, 661
- Prolog, 1136
- propagation modeling, 965
 - with LTI system, 966
- property class, 1059
- property grid, 1031
- property object, 1059
 - versus component, 1059

- property-centric, *see* object model
- pseudovector, 362
- PUBG, *see* genre, battle royale
- public, *see* C++
- pulse-code modulation, 948
- punning, *see* type punning
- pure component model, 1056
- Purify, 50
- push lock, 324
- PVS, *see* potentially visible set
- Python, 1134, 1140
 - method table, 1144
- quadratic probing, *see* hashing
- quadtree, 47, 694
- Quake C, 1138
- Quake Engine, 11, 31, 35, 45, 64, 1018, 1025
- Quantify, 50
- quantization, 136, *see* compression
- quaternion, 394, 405
 - concatenation, 398
 - conjugate, 397
 - conversion to matrix, 399
 - dual, 406
 - inverse, 396
 - product, 396
 - rotating vectors with, 397
- queue, 442
- quick time event, 735
- race, *see* data race
- race condition, 205, 258
- racing game, *see* genre
- Radiant, *see* game world editor
- radiosity, *see* lighting
- rag doll, 777, 886
- RAGE, *see* Rockstar Advanced Game Engine
- RAII, *see* design pattern
- random number, 412
 - Diehard tests, 413
 - KISS99, 414
 - linear congruential, 412
 - Mersenne Twister, 413
 - Mother of All, 413
 - PCC, 414
 - Xorshift, 414
- RAPID, 52
- rapid iteration, 1024
- rarefaction, 912
- rasterization, 664, 675
- Rational Purify, 101
- ray, 408
- ray cast, 1103
- ray tracing, *see* lighting
- RC filter, *see* filter
- RCA jack, *see* audio
- RCS, *see* version control
- RCU, *see* read-copy-update
- read-acquire, 311, 312, 319
- read-copy-update, 324
- read-modify-write, 259
- read-only data segment, *see* executable file, 151
- readers-writer lock, 324
- real-time strategy, *see* genre
- reality, *see* virtual reality
- record and play back, 50, 538
- rectangle invalidation, 525
- Redis, 67
- reduced instruction set, 223
- reference counting, 510, 1080
- referential integrity, 495, 498, 503, 516
- reflection, *see also* lighting, 1042, *see also* language
 - anisotropic, 634
 - diffuse, 634
 - reflectivity, 650
 - specular, 634
 - wave, 917
- refraction, *see* lighting, 917
- register, *see also* CPU
- registry (Microsoft Windows), 473
- relative velocity, *see* velocity
- relaxed memory order, *see* memory ordering semantics
- release build, *see* build configuration
- release fence, *see* memory ordering semantics
- release memory order, *see* memory ordering semantics
- relief mapping, *see* rendering
- render loop, 526

- render state, 691, 692
 - leak, 691
- render target, 664
- rendering
 - audio, *see* audio
 - billboard, *see* billboard
 - bump mapping, 699
 - deferred, 709
 - displacement mapping, 698
 - G-buffer, 709
 - parallax occlusion mapping, 698
 - relief mapping, 698
- rendering engine, 45
 - graphics device interface, 46
 - low-level renderer, 46, 47
 - render packet, 46
 - scene graph, 47, 622, 693, 697
- rendering equation, 622, 649
- rendering pipeline
 - application stage, 668, 687
 - asset conditioning stage, 668, 671
 - data transformation, 669
 - geometry processing stage, 668
 - GPU pipeline, 672
 - merging, 676
 - rasterization stage, 668
 - stream output, 674
 - tools stage, 668
 - triangle set-up, 675
 - triangle traversal, 675
- replacement policy, *see* cache
- repository, 69
- resource, 59
 - binary, 520
 - compiler, 502
 - composite, 516, 521
 - database, 494, 1035
 - dependencies, 502, 516
 - directory organization, 504
 - exporting, 499, 502
 - file formats, 507
 - GUID, 507
 - linker, 502
 - memory, 511
 - metadata, 494
 - registry, 508
 - sectioned files, 516
 - source asset, 59, 495, 501
- resource acquisition is initialization, *see* design pattern, RAI
- resource dependency, 222
- resource interchange file format, 952
- resource manager, 45, 481, 493
 - OGRE, 501
 - runtime, 503
 - Uncharted/TLOU, 498
 - Unreal, 496
 - XNA, 501
- response, 262
- restitution, *see* Newton's law of
- retargeting, *see* animation
- return address, 153
- reverb, 920, 922, 975
 - region, 967
 - tank, 979
- RIFF, *see* resource interchange file format
- right-hand rule, 371, 395
- rigid body dynamics, *see* physics
- ring buffer, 983
- RMS, *see* root mean square
- RMW, *see* read-modify-write
- Robin Hood hashing, *see* hashing
- Rockstar Advanced Game Engine, 33
- rodata segment, *see* executable file
- roll, 386
- root mean square, 914
- rope simulation, 818
- RTS, *see* genre
- RTTI, 1042
- run cycle, 734
- runtime scripting language, 1135
- runtime type identification, 1042
- S/PDIF, 954
- sampling, 683, *see also* animation, 737, 925, 948
 - depth conversion, 981
 - rate conversion, 981
- Sandbox, *see* game world editor
- saved games, 1042, 1078
- scalar, 221, 362
- scatter/gather, 546, 547
- scattering, *see* lighting

- SCCS, *see* version control
- scene graph, *see* rendering engine
- schema, 1066
 - inheritance, 1067
- Scheme, 477, 797, 1144
- scope, 152, 153
 - file, 152
 - lexical, 153
- scoped lock, 320
- Scream, *see* audio engine
- screen
 - aspect ratio, 659, 663
 - mapping, 663, 675
 - resolution, 659
- screenshot, 606
- scripting language, 58, 1040, 1134
 - class, 1153
 - data definition, 477, 1135
 - interface to native language, 1144
 - lambda, 1144
 - Lua, 1139
 - multitasking, 1154
 - object-oriented, 1152
 - Pawn, 1141
 - Python, 502, 1140
 - Quake C, 11, 1138
 - runtime, 1135
 - UnrealScript, 1138
- SDK, *see* software development kit
- SECAM, 534, 542, 663
- segment, *see* executable file
- select, 221
- semaphore, 275, 278, 492, 548, 555
 - binary, 276
- separating axis theorem, 837
- separating vector, 830
- sequential lock, 324
- sequential programming, 204, 544
- serial computer, 233
- serialization, 1063
- set associativity, *see* cache
- shader, 672
 - architecture, 677
 - Cg, 680
 - effect file, 670, 682
 - geometry, 674
 - High-Level Shading Language, 677
 - memory access, 678
 - OpenGL shader language, 680
 - pass, 682
 - pixel, 675
 - pixel shader, 46
 - register, 678
 - semantic, 680
 - shader model 4.0, 677
 - technique, 682
 - texture access, 679
 - texture sampler, 681
 - uniform declaration, 680
 - vertex, 629, 673
- shader resource table, 678
- shading, 633, 647
- shading language, 351
- shadow, 48, 655, 703
 - contact, 705
 - mapping, 703, 704
 - volume, 703
- Shannon-Nyquist sampling theorem, 948, 949
- shape, *see also* collision
 - sphere, 408
- shaping, *see* text rendering
- shared memory, *see* concurrency
- shared-exclusive lock, 324
- ship build, *see* build configuration
- shuffle, *see* SIMD instruction
- SI units, 912
- side-chain input, 990
- sign bit, *see* integer, *see also*
 - floating-point, 134
- signal, 912, 924
 - between threads, 276
 - continuous-time, 925, 948
 - discrete-time, 925, 948
 - manipulating, 925
 - periodic, 913, 938
 - processing theory, 783, 924
- signaled kernel object, 268, 276
- signed distance field, 717
- significant digits, *see* floating-point
- silhouette edge, 625, 689, 703

- SIMD, *see* single instruction multiple data
- SIMD unit, *see* GPU
- simplex, 840
- SIMT, *see* single instruction multiple thread
- simulation, 9, 51
 - agent-based, 9, 1041
 - discrete event, 1087
 - game genre, *see* genre
 - hard real-time, 10
 - interactive, 9, 525
 - physics, 10
 - real-time, 9, 525, 532, 1041
 - soft real-time, 10
 - temporal, 9
- single instruction multiple data, 160, 168, 170, 207, 221, 331, 336, 341, 348, 355, 393, 408, 413, 431, 548, 672, 678, 831
 - __m128, 332, 350
 - AltiVec, 336
 - horizontal add, 338
 - instruction, 334
 - lanes, 343
 - shuffle, 341, 342
 - vector float, 336, 350
 - VF32, 139
- single instruction multiple thread, 209, 331, 348, 355
- single instruction single data, 207
- single-step, *see* debugger
- singleton, *see* design pattern
- singularity function, 928
- sinusoid, 933
- SISD, *see* single instruction single data
- skeletal animation, *see* animation
- skeleton, 670
 - in memory, 728
- SketchUp, 669
- skinning, 62, 670, 750
 - to multiple joints, 754
 - weights, 635, 725, 750
- sky, 713
 - box, 714
 - dome, 714
- small memory allocator, 1076
- smart pointer, 440, 1042, 1079, 1080
- smoothing, 639
- SMP, *see* symmetric multiprocessing
- snapshot, 1112
- SoC, *see* system on a chip
- socket, *see* animation
- Softimage/XSI, 62, 669
- software development kit, 40
- software object model, *see* object model
- solid angle, 963
- sorting, 443
 - for rendering, 692
- sound, *see also* audio, 912
 - 2D, 955
 - 3D, 955
 - bank, 987
 - card, 974
 - clip, 955, 985, 998
 - cue, 986
 - dry, 920
 - fall-off of pressure, 917, 923, 957
 - portals, 972
 - pressure, 912, 914
 - pressure level, 915, 961
 - radiation patterns, 918
 - synthesis, 955, 975, 978, 995
 - wave, 913
 - wet, 921
- Sound Forge, 63
- Source engine, 33
- source file, *see* C++
- SourceSafe, *see* version control
- spatial hash, 693, 846, 1086
- spatial partitioning, 693
- spatialization, *see* audio
- spawner, 1065
 - pros and cons, 1069
- speakers, 943
 - speaker circle, 959
- spectral plot, 633
- Spectre exploit, 239
- specular, *see* lighting
- specular lighting, *see* lighting
- speculative execution, 218, *see* CPU
- speech, 997
- speed, 535
 - of sound, 913

- sphere hierarchy, *see* tree
- sphere map, 700
- spherical
 - coordinates, 360
 - harmonic basis function, 708
- spin lock, 271, 290, 294, 555
- spin-wait, *see* busy-wait
- SPL, *see* sound, pressure level
- spline, 624, 1033
 - B-spline, 624, 783
 - Bézier, 624
- split-screen, 995, 997
- spot light, *see* lighting
- spring, 884
- SPU, 545
- SQL Server, 498
- SQT transform, 405
- SRT, *see also* shader resource table
 - transform, 405, 729
- SSE, *see* streaming SIMD extensions
 - compiler intrinsics, 333
- stability (numerical), 863
- stack (data structure), 442
- stack frame, *see* call stack
- stage, *see* CPU, stage
- stall, 215
- stance variation, 772
- standard C library, *see* C standard
 - library
- standard C++ library, *see* C++
- standard template library, 40, 41, 442, 448
 - `std::list`, 41
 - `std::string`, 457, 465
 - `std::vector`, 41
- start-up and shut-down
 - construct on demand, 419
 - engine subsystem, 417
 - manual, 420
 - OGRE, 422
 - Uncharted/TLOU*, 424
- starvation, 284
- state caching, 1100
- static
 - allocation, *see* allocation
 - assertion, *see* assertion
 - lights, *see* lighting
 - variable, 149, 157
- stencil buffer, 664, 703
 - testing, 676
- stereo, 944
- stinger, 1011
- STL, *see* standard template library
- stochastic propagation modeling, 972
- strategy game, *see* genre
- streaming
 - audio streaming, 988
 - dialog streaming, 998
 - level streaming, 1040
- streaming SIMD extensions, 331
- strict aliasing, 143
- string, 456
 - class, 457, 465
 - database, 467
- string id, 459, 1138, 1151
- strip, *see* triangle
- struct of arrays, 1060
- structured bindings, 115
- studio, 5
 - first-party developer, 8
- subdivision surface, 624
- subnormal, *see* floating-point
- Subversion, *see* version control
- subwoofer, *see* low-frequency effects
- superposition, 919, 926
- superscalar, *see* CPU
- surface, 622, 623
 - visual properties, 622, 632
- surround sound, 944
- sweep and prune algorithm, 847
- SWIFT, 52
- symbolic link, 494
- symmetric multiprocessing, 227
- synchronization point, 1106, 1113
- synergistic processing unit, *see* SPU
- syntax tree, 792
- synthesizer, 995
- system, *see* linear time-invariant system
- system on a chip, 678
- T&L, *see* hardware T&L
- table (data structure), 1139
- Tagged Image File Format, 642
- Targa, 642

- target hardware, 38
- Target Manager, 590
- task decomposition, 544
- task parallelism, *see* parallelism
- taxonomy, 1048
- tearing, 537, 538, 663
- technical requirements checklist, 587
- template metaprogramming, 118
- temporal coherency, 537, 846
- temporal multithreading, 234
- temporary register, *see* GPU
- terrain, 714
 - height field, 60, 714, 1029
- tessellation, 625, 674
 - dynamic, 626, 714
- test-and-set instruction, 294
- testability, 87
- TeX, 1136
- texel, 639
 - density, 643, 644
- text rendering, *see also* signed distance
 - field, 716
 - shaping, 717
- text segment, *see* executable file
- texture, 46, 121, 639, 670, 678, 679
 - 1D, 641
 - 3D, 701
 - addressing modes, 641
 - albedo map, 640
 - animated, 52, 723
 - compression, 642
 - coordinates, 641
 - cube map, 700
 - diffuse map, 640
 - environment map, 48, 700, 706
 - filtering, 645, 679
 - formats, 642
 - gloss map, 699
 - heightmap, 698
 - light map, 48, 653, 672
 - normal map, 698
 - rendering to, 680, 706
 - scrolling, 691
 - shadow map, 703
 - specular map, 699
 - specular power map, 699
 - sphere map, 700
- thread, 204, *see also* GPU
 - affinity, 227, 248
 - dependencies, 548, 1105
 - group, *see* GPU
 - join, 243, 554, 557
 - priority, 248
 - safety, 288, 321, 1102
 - synchronization primitive, 265, 267, 555
 - user-level, 253, 1102, 1110
- throughput, *see* pipeline
- timbre, 920
- time
 - abstract timeline, 532
 - clock, 532
 - clock drift, 540
 - clock variable, 540
 - delta, 535–537, 572
 - domain, 924, 938, 974
 - floating-point, 541
 - game, 525, 532
 - high-resolution timer, 539
 - index, 735
 - local, 533
 - measuring, 539, 543
 - of impact, 845
 - scaling, 739
 - shift, 925
 - units, 540
- time-invariant, 926
- time-slicing, *see also* multitasking, 208, 234, 356, 545
- timeline
 - global, 739
 - local, 735
- TLB, *see* translation lookaside buffer
- TOI, *see* time of impact
- Tokamak, 825
- Tombstone Engine, 35
- tone mapping, 702
- tool, 59, 61, 669
- tools pipeline, *see* asset conditioning
 - pipeline
- torque, 373, 867
 - in three dimensions, 873
- Torque Engine, 36
- TOSLINK, 954

- transaction, 287, 328
 - committing, 328
- transformation, 376
 - and lighting, 672
 - coordinate axes, 384, 390
 - matrix, 376
 - rotation, 376, 382, 403
 - scale, 364, 376, 383
 - translation, 376, 382
- transistor, 946
- transition, *see* animation
- translation unit, *see* C++, 148
- translucency, 623
- transmission, *see* lighting
- transparency, 623
- transport model, *see* lighting
- transpose
 - matrix, 339
- tree, 442, *see also* blend tree
 - binary space partitioning tree, 47, 695
 - bounding sphere, 695, 846
 - kd-tree, 47, 695
 - octree, 47, 694
 - quadtree, 47, 694
 - sphere hierarchy, 47
- triangle, 625
 - area of, 371
 - face normal, 627
 - fan, 629, 630
 - indexed list, 628
 - list, 628, 630
 - mesh, *see* mesh
 - occlusion, 664
 - strip, 629, 630
 - winding order, 627
- triangulation, 625
- trilinear, *see* texture, filtering
- TRS jack, *see* audio
- TrueAxis, 824, 825
- TTY, 590
- tunneling, 843
- turbo button, 536
- turn-based strategy, *see* genre
- two's complement, *see* integer
- type punning, 143
- UAA, *see* Universal Audio Architecture
- UDN, 11, *see* Unreal Developer Network
- ULP, *see* floating-point
- umbra, 655
- UNC, 483
- Unicode, 462
 - UTF-16, 463
 - UTF-32, 463
 - UTF-8, 463
- Unified Modeling Language, 107
- unit impulse, 928
- unit in the last place, *see* floating-point
- Unity, 35
- Universal Audio Architecture, 992
- universal serial bus, *see* USB
- Unreal Developer Network, 33
- Unreal Engine, 11, 32, 35, 45, *see also*
 - audio engine
 - Blueprints, 32, 1133
- Unreal Tournament 2004, 32
- UnrealEd, 496, 1035, *see* game world
 - editor
- UnrealScript, 1138
- unresolved symbol error, *see* compiler, 146
- USB, 570, 954
- user-defined literals, *see* C++
- user-level thread, *see* thread
- UTF-x, *see* Unicode
- V-Collide, 52
- VAG, *see* audio, file formats
- Valgrind, 50, 102
- variable
 - class static, 157
 - global, 151
 - local, 153
 - member, 156
 - static, 151
- variant, 1008, 1118, 1145
- vector, 362, *see also* perpendicular
 - addition, 364
 - arithmetic, 365
 - basis, 363, 390
 - collinear, 369
 - cross product, 367, 370

- direction, 363, 381
 - dot product, 367
 - front, left and up, 385, 631
 - magnitude, 365
 - multiplication by scalar, 364
 - normal, 367, 369, 392, 638
 - normalization, 367
 - perpendicular, 635
 - position, 363
 - projection, 368
 - quaternion form, 397
 - squared magnitude, 367
 - STL, 442
 - subtraction, 365
 - unit, 363, 367
- vector processing unit, 168, 208, 224, *see also* GPU, SIMD unit, 350, 672
- vector unit (PS2), 224
- vectorization, 168, 337, 344, 348, 548
- vehicle engine sound, 996
- velocity, 366, 528, 535
 - angular, 867
 - animation, 758
 - linear, 858
 - relative, 923, 957, 973
 - screen space, 719
- version control, 69
 - Alienbrain, 70, 494
 - assets, 493
 - checking in and out, 75
 - ClearCase, 70
 - committing, 75
 - CVS, 70
 - deleting files, 78
 - diff, 75
 - exclusive check-out, 76
 - git, 70
 - history, 74
 - hosting, 72
 - locking, 76
 - merging, 76
 - multiple check-out, 76
 - Perforce, 70, 493
 - RCS, 70
 - rebasing, 70
 - repository, 72
 - resources, 493
 - SCCS, 70
 - SourceSafe, 70
 - submitting, 75
 - Subversion, 70
 - three-way merge, 76, 102
 - TortoiseSVN, 73
 - updating, 74
- vertex, 635
 - attribute interpolation, 637, 662, 675
 - attributes, 635
 - binormal, 635
 - bitangent, 635
 - cache optimization, 629, 679
 - formats, 636
 - normal, 635
 - tangent, 635
- vertex buffer, 628
- vertical blanking interval, 538, 663
- view volume, 658, 674
 - far plane, 658
 - frustum, 411, 658, 674
 - near plane, 658
- viewing direction, *see* lighting
- viewport, 46
- vignette, 719
- virtual machine, 317, 1135, 1144
- virtual reality, 27, 35
- visibility determination, 47, 157
- Visual Basic, 1134
- visual effects, 48, 64
- VLIW, *see* CPU
- voice, *see* audio
- VoIP, 570
- volatile, 302, 318, 326
- volume control, 946, 979, 983
- Voodoo, 672
- VPU, *see* vector processing unit
- VR, *see* virtual reality
- VTune, 50
- VU0 (PS2), 224
- Vulkan, 41, 46, 692
- w* buffer, 664, 666, 704
- wait-free algorithm, 290
- wait-free consensus problem, *see* consensus problem

- walk cycle, 734
- warp, *see* GPU
- WASAPI, *see* Windows Audio Session API
- water, 626, 632, 647, 648, 673, 706, 710, 715, 910, 1017, 1029, 1044
- WAV, *see* audio, file formats
- wave
 - longitudinal, 917
 - propagation, 917
 - transverse, 917
- wavefront, *see* GPU
- wavelength, 633, *see also* lighting, 914
- WDM, *see* Windows Driver Model
- weighted average, 375, 635, 676, 765, 788
- wet sound, *see* sound
- whole number, 132
- Wiimote, 53
- Windows
 - Audio Session API, 993
 - Bitmap, 642
 - Driver Model, 992
 - Media Video (WMV), 1162
- WMA, *see* audio, file formats
- WMV, 1162
- woofer, *see* low-frequency effects
- work, 875
- world, *see* game world
- world-space texel density, 644
- write
 - back, 196, *see also* CPU, stage, 306, 307, 309
 - release, 311, 312, 319
 - through, 196, 306
- Wwise, 995

- X3DAudio, *see* audio engine
- XACT, *see* audio engine
- XAudio2, *see* audio engine
- Xbox Live, 34, 473, 481, 487
- XML, 1136
- XNA Game Studio, 34, 993

- Yake, 36
- yaw, 386
- yielding
 - coroutine, 253, 554, 1110
 - fiber, 250, 251, 554
 - thread's timeslice, 234, 241, 244, 246, 250
- z bias, 712
- z buffer, 704
- z fighting, 666, 712
- ZBrush, 62
- ZIP archive, *see* archive file



Plate I. *Overwatch* by Blizzard Entertainment (Xbox One, PlayStation 4, Windows). (See Figure I.2 on page 14.)



Plate II. *Jak II* by Naughty Dog (Jak, Daxter, Jak and Daxter, and Jak II © 2003, 2013/™ SIE. Created and developed by Naughty Dog, PlayStation 2). (See Figure I.3 on page 16.)



Plate III. *Gears of War 4* by The Coalition (Xbox One). (See Figure I.4 on page 17.)



Plate IV. *Tekken 3* by Namco (PlayStation). (See Figure I.5 on page 18.)



Plate V. *Injustice 2* by NetherRealm Studios (PlayStation 4, Xbox One, Android, iOS, Microsoft Windows). (See Figure 1.6 on page 19.)



Plate VI. *Gran Turismo Sport* by Polyphony Digital (PlayStation 4). (See Figure 1.7 on page 20.)



Plate VII. *Age of Empires* by Ensemble Studios (Windows). (See Figure 1.8 on page 21.)



Plate VIII. *Total War: Warhammer 2* by Creative Assembly (Windows). (See Figure I.9 on page 22.)



Plate IX. *World of Warcraft* by Blizzard Entertainment (Windows, MacOS). (See Figure I.10 on page 23.)



Plate X. *Destiny 2* by Bungie, © 2018 Bungie Inc. (Xbox One, PlayStation 4, PC). (See Figure I.II on page 24.)



Plate XI. *LittleBigPlanet*™ 2 by Media Molecule, © 2014 Sony Interactive Entertainment (PlayStation 3). (See Figure I.12 on page 25.)



Plate XII. *Dreams* by Media Molecule, © 2017 Sony Interactive Entertainment (PlayStation 4). (See Figure I.13 on page 26.)



Plate XIII. *Minecraft* by Markus "Notch" Persson / Mojang AB (Windows, MacOS, Xbox 360, PlayStation 3, PlayStation Vita, iOS).
(See Figure I.14 on page 26.)



Plate XIV. *Accounting* by Squanchtendo and Crows Crows Crows (HTC Vive). (See Figure 1.15 on page 29.)



Plate XV. A scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) rendered without textures. (See Figure II.20 on page 647.)



Plate XVI. The same scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) with only diffuse textures applied. (See Figure 11.21 on page 648.)



Plate XVII. Scene from *The Last of Us: Remastered* (© 2014™ SIE. Created and developed by Naughty Dog, PlayStation 4) with full lighting. (See Figure 11.22 on page 648.)



Plate XVIII. The flashlight in *Luigi's Mansion* by Nintendo (Wii) is composed of numerous visual effects, including a cone of translucent geometry for the beam, a dynamic spot light to cast light into the scene, an emissive texture on the lens and camera-facing cards for the lens flare. (See Figure 11.30 on page 656.)

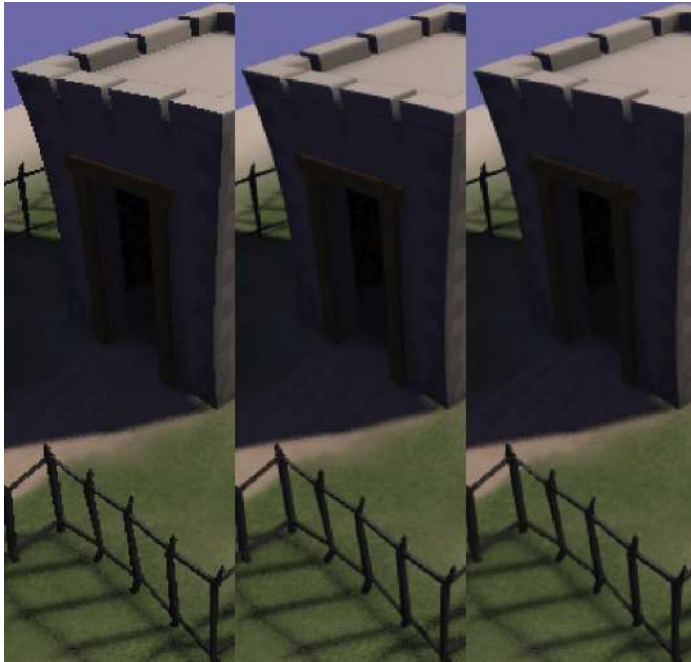


Plate XIX. Left: No antialiasing. Center: 4× MSAA. Right: Nvidia's FXAA, preset 3. Image from Nvidia's FXAA white paper by Timothy Lottes (<http://bit.ly/1mlzCTv>). (See Figure II.45 on page 683.)



Plate XX. This screenshot from EA's *Fight Night Round 3* shows how a gloss map can be used to control the degree of specular reflection that should be applied to each texel of a surface. (See Figure 11.55 on page 701.)



Plate XXI. Mirror reflections in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) implemented by rendering the scene to a texture that is subsequently applied to the mirror's surface. (See Figure 11.59 on page 707.)

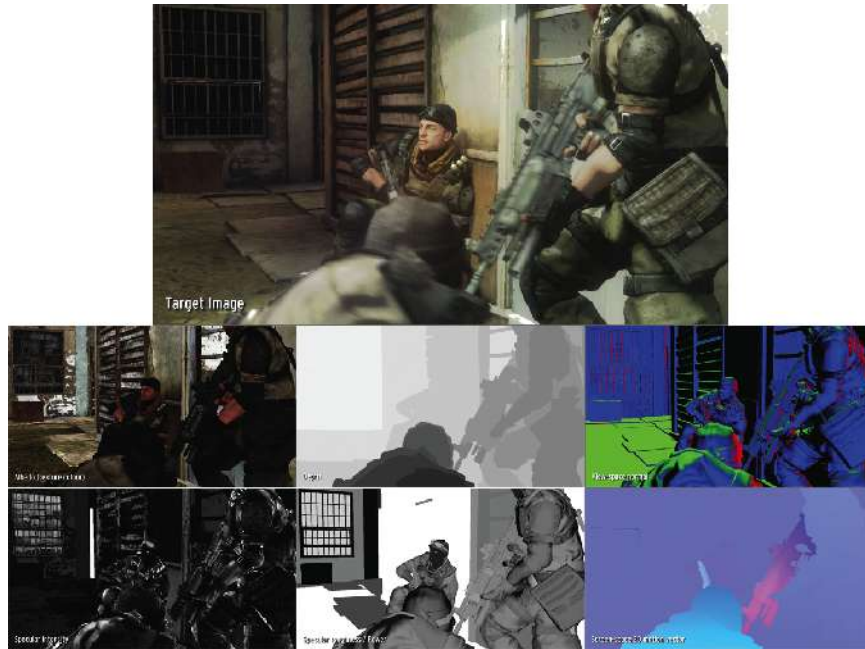


Plate XXII. Screenshots from *Killzone 2* by Guerrilla Games, showing some of the typical components of the G-buffer used in deferred rendering. The upper image shows the final rendered image. Below it, clockwise from the upper left, are the albedo (diffuse) color, depth, view-space normal, screen-space 2D motion vector (for motion blurring), specular power and specular intensity. (See Figure 11.62 on page 710.)



Plate XXIII. Flame, smoke and bullet tracer particle effects in *Uncharted 3: Drake's Deception* (© 2011/™ SIE. Created and developed by Naughty Dog, PlayStation 3). (See Figure 11.63 on page 712.)



Plate XXIV. Parallax-mapped decals from *Uncharted 3: Drake's Deception* (© 2011/™ SIE. Created and developed by Naughty Dog, PlayStation 3). (See Figure 11.64 on page 713.)

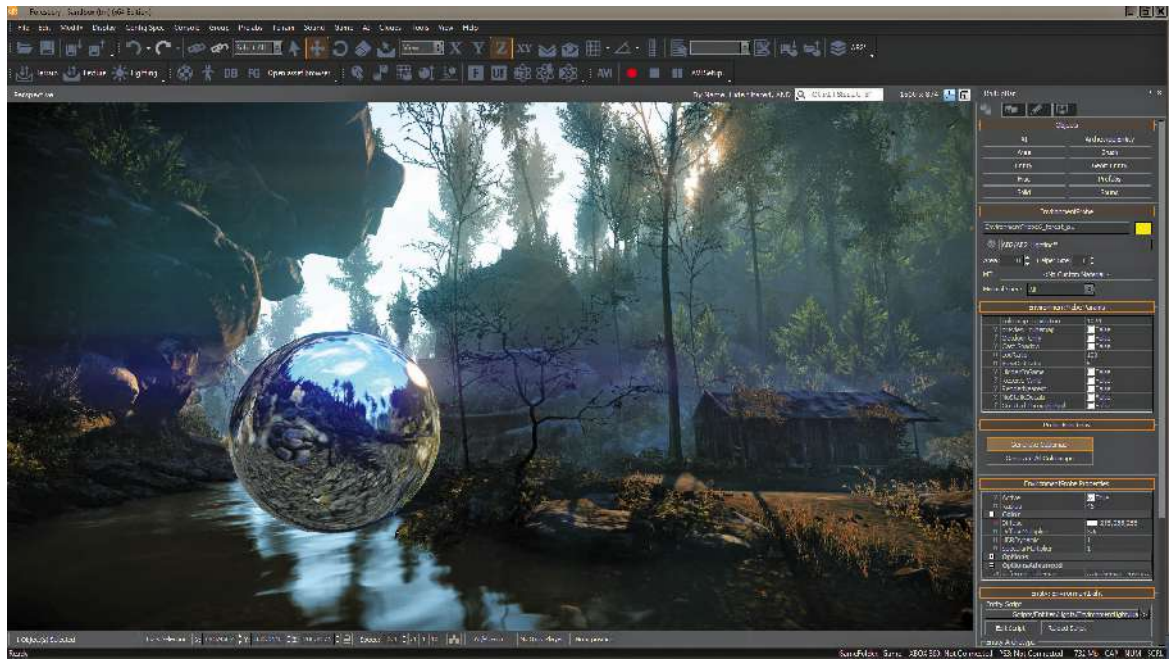


Plate XXV. The Sandbox editor for CRYENGINE. (See Figure 15.6 on page 1028.)

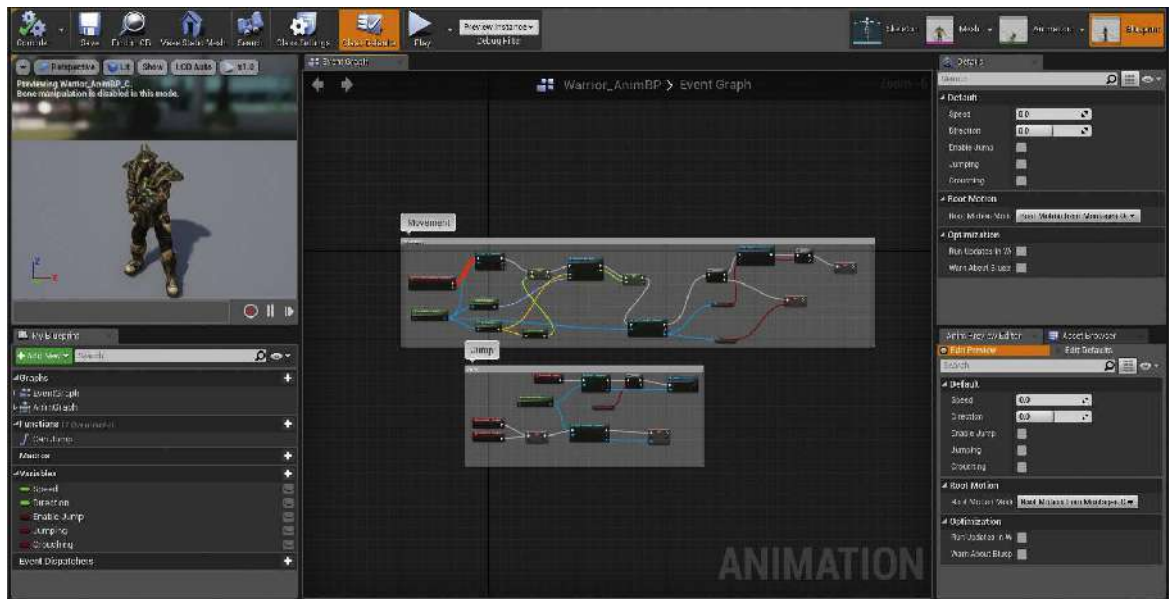


Plate XXVI. The Unreal Engine 4 animation Blueprints editor. (See Figure 12.58 on page 801.)